第 2 周：凸优化与 KKT 条件

从 NP-Hard 到多项式时间：凸性的魔力

目录

1 回顾：从复杂度到结构 **4**

2 凸集与凸函数 **5**
2.1 凸集的定义 5
2.2 凸函数的定义 6

3 凸性判定 **7**
3.1 一阶条件 7
3.2 二阶条件 8

4 凸的礼物：局部即全局 **10**

5 凸优化的标准形式 **11**
5.1 线性规划 (LP) 的例子 12
5.2 二次规划 (QP) 的例子 13
5.3 二阶锥规划 (SOCP) 的例子 15
5.4 半定规划 (SDP) 的例子 16

6 Lagrangian 与对偶函数 **18**

7 弱对偶与强对偶 **19**

8 KKT 条件 **20**

9 对偶的经济学解释 **22**

10 机器学习视角：凸优化的现代应用 **26**

11 MDP 与线性规划 ★ **28**
11.1 具体例子：机器维护问题 30
11.2 Bellman 方程 31
11.3 MDP 的线性规划形式 32
11.4 LP 对偶与占用度量 35

12 GPU 改变现代优化 **37**

13 实验 **40**
13.1 实验 2a：CVXPY 求解 SVM 对偶 40
13.2 实验 2b：GPU vs CPU 凸优化 41

14 本周总结 **44**

15 思考题 **46**

本讲导读

内容层次

核心内容（课堂必讲，约 90 分钟）：

- 凸集、凸函数的定义与判定（一阶/二阶条件）
- 凸优化的标准形式，LP/QP 例子
- Lagrangian、对偶函数、弱对偶
- KKT 条件（四个条件 + 直觉）
- 实验 2a/2b

重要证明（标注 ★，选讲或自学）：

- 一阶条件、二阶条件的证明
- KKT 条件的推导
- Logistic 回归的凸性证明

进阶内容（标注 ★，适合优秀学生）：

- MDP 与线性规划：序贯决策问题的优化视角
- Bellman 方程 $\to$ LP 不等式
- LP 对偶 $\leftrightarrow$ 值函数 vs 占用度量

扩展内容（自学材料）：

- SOCP、SDP 的详细例子
- 机器学习视角的深入讨论
- 影子价格的完整推导

数学记号

记号	含义
$\nabla f(x)$	梯度（列向量）
$\nabla^2 f(x)$	Hessian 矩阵
$A \succeq 0$	A 半正定（所有特征值 ≥ 0）
$A \succ 0$	A 正定（所有特征值 > 0）
$\langle A, B \rangle$	矩阵内积 $= \mathrm{tr}(A^T B)$
p^*	原问题最优值
d^*	对偶问题最优值
λ, ν	Lagrange 乘子

1 回顾：从复杂度到结构

第 1 周的核心结论

问题的难度层级：

P（高效）	NP	NP-Hard	不可判定
排序、最短路	SAT、TSP	一般优化	停机问题

→ 难度

关键问题：一般优化是 NP-Hard，我们怎么办？

从 NP-Hard 逃脱的方法

方法 1：接受近似解（启发式算法）

方法 2：利用问题的**特殊结构**

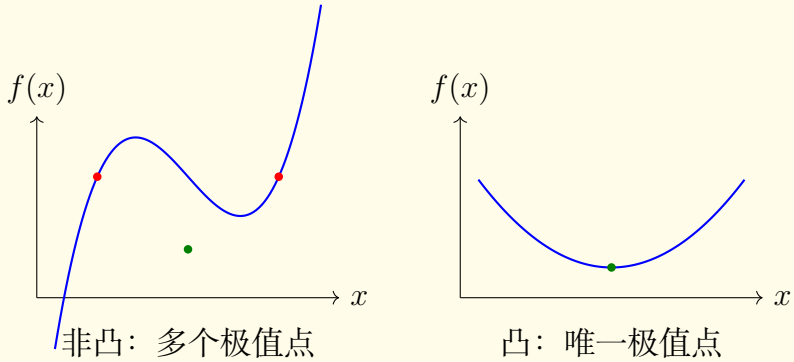
结构	复杂度	例子
无结构（一般）	NP-Hard	一般非线性优化
凸	**P**	**凸优化（本周重点）**
线性	P	线性规划
图结构	取决于具体问题	最短路、最小生成树

本周主题：**凸性**是最重要的“好结构”

直觉：凸性为什么重要？

优化的终极梦想：给我一个函数，我想找它的最小值

噩梦：函数有很多局部最小值，你找到的可能不是全局最优



非凸：多个极值点　　凸：唯一极值点

凸的礼物：局部最优 = 全局最优！

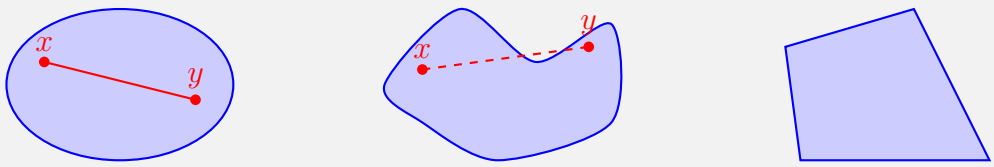
2 凸集与凸函数

2.1 凸集的定义

凸集：几何定义

定义 2.1 (凸集)**.** 集合 $C \subseteq \mathbb{R}^n$ 是**凸集**，当且仅当：
对任意 $x, y \in C$ 和任意 $\theta \in [0,1]$，有 $\theta x + (1-\theta)y \in C$

直觉：集合内任意两点的连线段都完全在集合内



凸集 ✓　　非凸集 ×　　多面体（凸）

常见的凸集

1. **超平面**：$\{x : a^T x = b\}$
2. **半空间**：$\{x : a^T x \le b\}$
3. **多面体**（半空间的交集）：$\{x : Ax \le b\}$
4. **球**：$\{x : \|x - x_0\| \le r\}$
5. **椭球**：$\{x : (x-x_0)^T P^{-1}(x-x_0) \le 1\}$，$P \succ 0$
6. **锥**：$\{x : \|x\| \le t\}$（二阶锥）

7. 半正定锥：$\{X \in \mathbb{R}^{n\times n} : X = X^T, X \succeq 0\}$
关键性质：凸集的交集仍是凸集

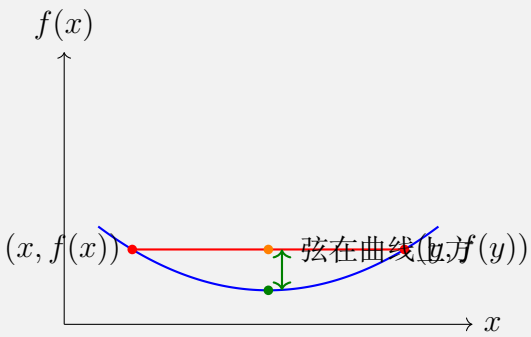
2.2 凸函数的定义

凸函数：几何定义

定义 2.2 (凸函数)**.** 函数 $f : \mathbb{R}^n \to \mathbb{R}$ 是**凸函数**，当且仅当：
对任意 x, y 和任意 $\theta \in [0, 1]$：

$$f(\theta x + (1-\theta)y) \le \theta f(x) + (1-\theta) f(y)$$

直觉：函数图像上任意两点连线在函数图像的**上方**



Jensen 不等式

定理 2.3 (Jensen 不等式)**.** 如果 f 是凸函数，$x_1, \ldots, x_k$ 是任意点，$\theta_1, \ldots, \theta_k \ge 0$ 且 $\sum_i \theta_i = 1$，则：

$$f\left(\sum_{i=1}^{k} \theta_i x_i\right) \le \sum_{i=1}^{k} \theta_i f(x_i)$$

直觉："加权平均后求函数值" $\le$ "函数值的加权平均"
特例（期望形式）：如果 X 是随机变量，f 是凸函数：

$$f(\mathbb{E}[X]) \le \mathbb{E}[f(X)]$$

应用：

- $f(x) = x^2$ 凸 $\Rightarrow (\mathbb{E}[X])^2 \le \mathbb{E}[X^2]$，即 $\mathrm{Var}(X) \ge 0$
- $f(x) = -\log x$ 凸 $\Rightarrow -\log(\mathbb{E}[X]) \le \mathbb{E}[-\log X]$（AM-GM 不等式）

常见的凸函数

一元凸函数：

- $x^2, |x|, e^x, x\log x \quad (x>0)$，$-\log x \quad (x>0)$

多元凸函数：

- 仿射函数：a^Tx+b（既凸又凹）
- 范数：$\|x\|_1, \|x\|_2, \|x\|_\infty$
- 二次函数：x^TPx，当 $P \succeq 0$
- Log-sum-exp：$\log(e^{x_1}+\cdots+e^{x_n})$
- 最大值函数：$\max\{x_1,\ldots,x_n\}$

保持凸性的操作：

- 非负加权和：$\sum_i \alpha_i f_i$，$\alpha_i \geq 0$
- 复合仿射：$f(Ax+b)$
- 逐点最大：$\max\{f_1(x), f_2(x)\}$

3 凸性判定

如何判断函数是否凸？

方法 1：直接验证定义（通常很难）
方法 2：一阶条件（需要可微）
方法 3：二阶条件（需要二次可微）
方法 4：证明它是凸函数的组合

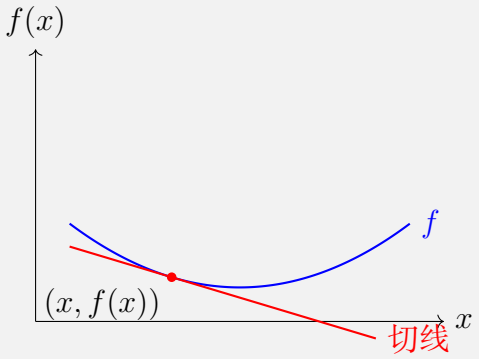
3.1 一阶条件

一阶条件：切线在下方

定理 3.1 (凸性的一阶条件)**.** 设 f 可微。f 是凸函数当且仅当对所有 x, y：

$$f(y) \geq f(x) + \nabla f(x)^T(y-x)$$

几何意义：函数图像始终在其切平面的**上方**



推论：如果 $\nabla f(x^*) = 0$，则 x^* 是**全局最小值**！

一阶条件的证明 ★

证明（$\Rightarrow$）：设 f 凸，要证 $f(y) \geq f(x) + \nabla f(x)^T(y-x)$
由凸性定义，对 $\theta \in (0,1)$：

$$f(x + \theta(y-x)) \leq (1-\theta)f(x) + \theta f(y)$$

移项：

$$f(x + \theta(y-x)) - f(x) \leq \theta(f(y) - f(x))$$

除以 θ：

$$\frac{f(x + \theta(y-x)) - f(x)}{\theta} \leq f(y) - f(x)$$

令 $\theta \to 0^+$，左边趋于方向导数 $\nabla f(x)^T(y-x)$：

$$\nabla f(x)^T(y-x) \leq f(y) - f(x)$$

即 $f(y) \geq f(x) + \nabla f(x)^T(y-x)$ □

3.2 二阶条件

二阶条件：Hessian 半正定

定理 3.2 (凸性的二阶条件). *设 f 二次可微。f 是凸函数当且仅当对所有 x：*

$$\nabla^2 f(x) \succeq 0$$

即 Hessian 矩阵处处半正定。

回忆：矩阵 H 半正定 $\Leftrightarrow$ 所有特征值 $\geq 0 \Leftrightarrow v^T H v \geq 0$ 对所有 v
直觉：二阶导数处处非负 $\Rightarrow$ 函数“向上弯曲”

二阶条件的证明 ★

证明（$\Rightarrow$）：设 f 凸，要证 $\nabla^2 f(x) \succeq 0$
由一阶条件，对任意 y：

$$f(y) \geq f(x) + \nabla f(x)^T (y - x)$$

同时，交换 x, y 的角色：

$$f(x) \geq f(y) + \nabla f(y)^T (x - y)$$

两式相加：

$$0 \geq (\nabla f(x) - \nabla f(y))^T (y - x) = -(\nabla f(y) - \nabla f(x))^T (y - x)$$

即 $(\nabla f(y) - \nabla f(x))^T (y - x) \geq 0$（梯度是**单调**的）
设 $y = x + tv$，$t > 0$：

$$(\nabla f(x + tv) - \nabla f(x))^T v \geq 0$$

除以 t，令 $t \to 0$：

$$v^T \nabla^2 f(x) v \geq 0$$

对所有 v 成立，故 $\nabla^2 f(x) \succeq 0$ □

例子：二次函数

函数：$f(x) = \frac{1}{2} x^T P x + q^T x + r$
梯度：$\nabla f(x) = Px + q$
Hessian：$\nabla^2 f(x) = P$
凸性：f 是凸函数 $\Leftrightarrow P \succeq 0$
强凸性：f 是强凸函数 $\Leftrightarrow P \succ 0$

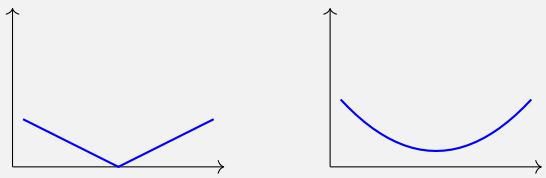
强凸性

定义 3.3 (强凸函数)**.** 函数 f 是 m-**强凸**的，如果对所有 x：

$$\nabla^2 f(x) \succeq mI, \quad m > 0$$

等价定义：$f(x) - \frac{m}{2}\|x\|^2$ 是凸函数
性质：

- 强凸函数有**唯**一的全局最小值
- 梯度下降收敛更快（线性收敛 vs 次线性收敛）



4 凸的礼物：局部即全局

凸优化的核心定理

定理 4.1 (局部最优 = 全局最优). 对于凸优化问题：

$$\min_x f(x) \quad s.t. \quad x \in C$$

其中 f 是凸函数，C 是凸集。
如果 x^* 是**局部最优解**，则 x^* 是**全局最优解**。

证明：反证法。假设存在 $y \in C$ 使得 $f(y) < f(x^*)$。
由凸性，对任意 $\theta \in (0,1)$：

$$f(\theta y + (1-\theta)x^*) \leq \theta f(y) + (1-\theta) f(x^*) < f(x^*)$$

当 $\theta \to 0$ 时，$\theta y + (1-\theta)x^* \to x^*$，矛盾于 x^* 是局部最优。

直觉：为什么这个定理如此重要？

非凸优化的噩梦：

- 梯度下降可能陷入局部最优
- 不知道找到的解有多好
- 可能需要多次随机重启

凸优化的美梦：

- 任何下降方法都能找到全局最优
- 有收敛性保证
- 一次运行就够

结论：证明问题是凸的，是优化研究的第一步！

5 凸优化的标准形式

凸优化的标准形式

标准形式：

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \le 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

凸优化的要求：

- 目标函数 f 是**凸函数**
- 不等式约束 g_i 是**凸函数**
- 等式约束 h_j 是**仿射函数**（$h_j(x) = a_j^T x - b_j$）

注意：等式约束只能是仿射的！
原因：$h(x) = 0$ 等价于 $h(x) \le 0$ 且 $-h(x) \le 0$，要同时凸只能是仿射

常见的凸优化问题

1. 线性规划 (LP)：

$$\min_x c^T x \quad \text{s.t.} \quad Ax \le b, \quad Cx = d$$

2. 二次规划 (QP)：

$$\min_x \frac{1}{2} x^T P x + q^T x \quad \text{s.t.} \quad Ax \le b$$

$(P \succeq 0)$

3. 二阶锥规划 (SOCP)：

$$\min_x c^T x \quad \text{s.t.} \quad \|A_i x + b_i\|_2 \le c_i^T x + d_i$$

4. 半定规划 (SDP)：

$$\min_X \langle C, X \rangle \quad \text{s.t.} \quad \langle A_i, X \rangle = b_i, \quad X \succeq 0$$

包含关系：LP $\subset$ QP $\subset$ SOCP $\subset$ SDP $\subset$ 凸优化

5.1 线性规划 (LP) 的例子

LP 例子 1：生产计划

场景：工厂生产两种产品 A 和 B

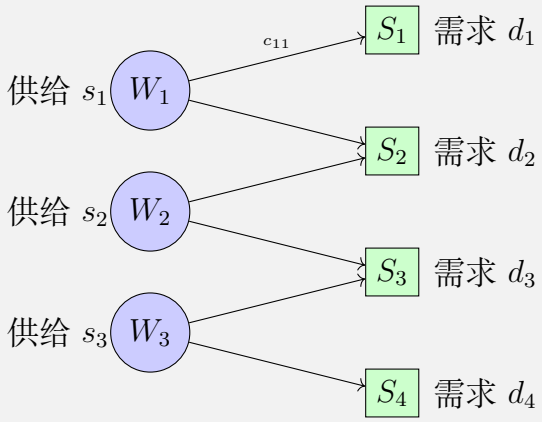
- 产品 A：利润 40 元，需要 2 小时机器时间，3 kg 原料
- 产品 B：利润 30 元，需要 1 小时机器时间，2 kg 原料
- 约束：每天最多 100 小时机器时间，120 kg 原料

建模：设 x_1, x_2 为两种产品的产量

$$\begin{aligned} \max_{x_1,x_2} \quad & 40x_1 + 30x_2 \\ \text{s.t.} \quad & 2x_1 + x_2 \le 100 \quad \text{（机器时间）} \\ & 3x_1 + 2x_2 \le 120 \quad \text{（原料）} \\ & x_1, x_2 \ge 0 \end{aligned}$$

LP 例子 2：运输问题

场景：3 个仓库向 4 个商店配送货物，最小化运输成本



建模：设 x_{ij} 为从仓库 i 到商店 j 的运输量

$$\begin{aligned} \min \quad & \sum_{i,j} c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_j x_{ij} \le s_i \quad \text{（供给约束）} \\ & \sum_i x_{ij} \ge d_j \quad \text{（需求约束）} \\ & x_{ij} \ge 0 \end{aligned}$$

LP 例子 3：饮食问题

场景：以最低成本满足每日营养需求

食物	蛋白质 (g)	碳水 (g)	脂肪 (g)	价格 (元)
牛奶	8	12	8	3
鸡蛋	13	1	11	2
面包	4	52	1	2
每日需求	≥ 55	≥ 300	≥ 50	

建模：设 x_1, x_2, x_3 为三种食物的份数

$$\begin{aligned} \min \quad & 3x_1 + 2x_2 + 2x_3 \\ \text{s.t.} \quad & 8x_1 + 13x_2 + 4x_3 \geq 55 \\ & 12x_1 + x_2 + 52x_3 \geq 300 \\ & 8x_1 + 11x_2 + x_3 \geq 50 \\ & x_1, x_2, x_3 \geq 0 \end{aligned}$$

5.2 二次规划 (QP) 的例子

QP 例子 1：Markowitz 投资组合

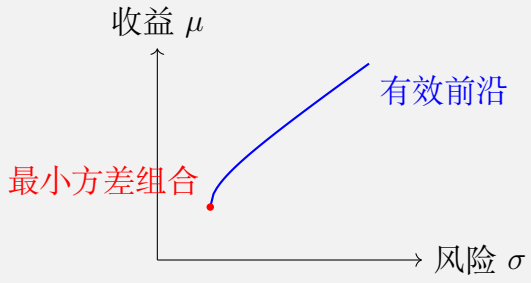
场景：投资 n 只股票，平衡收益与风险

- μ_i：股票 i 的预期收益率
- Σ：收益率的协方差矩阵（风险）
- x_i：投资在股票 i 的比例

建模：

$$\begin{aligned} \min_x \quad & \frac{1}{2} x^T \Sigma x \quad \text{（最小化风险）} \\ \text{s.t.} \quad & \mu^T x \geq r_{\min} \quad \text{（保证最低收益）} \\ & \mathbf{1}^T x = 1 \quad \text{（投资比例和为 1）} \\ & x \geq 0 \quad \text{（不允许卖空）} \end{aligned}$$

有效前沿：改变 $r_{\min}$，得到收益-风险权衡曲线



QP 例子 2：支持向量机 (SVM)

场景：二分类问题，找最大间隔超平面

原始问题：

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2}\|w\|^2 \\ \text{s.t.} \quad & y_i(w^T x_i + b) \geq 1, \quad \forall i \end{aligned}$$

这是一个 QP！目标函数是 w 的二次函数，约束是线性的。

软间隔 SVM（允许误分类）：

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|^2 + C\sum_i \xi_i \\ \text{s.t.} \quad & y_i(w^T x_i + b) \geq 1 - \xi_i \\ & \xi_i \geq 0 \end{aligned}$$

QP 例子 3：岭回归

场景：带正则化的线性回归

普通最小二乘：$\min_w \|Xw - y\|^2$

问题：当特征多、样本少时，过拟合严重

岭回归：

$$\min_w \|Xw - y\|^2 + \lambda\|w\|^2$$

展开：

$$\min_w w^T(X^T X + \lambda I)w - 2y^T Xw + \text{const}$$

这是 QP！$P = 2(X^T X + \lambda I) \succ 0$（强凸）

闭式解：$w^* = (X^T X + \lambda I)^{-1} X^T y$

5.3 二阶锥规划 (SOCP) 的例子

SOCP 例子 1：鲁棒线性规划

场景：参数有不确定性的 LP

普通 LP：$\min c^T x$ s.t. $a_i^T x \le b_i$

问题：如果 a_i 有误差怎么办?

假设 $a_i \in \mathcal{E}_i = \{\bar{a}_i + P_i u : \|u\| \le 1\}$（椭球不确定集）

鲁棒约束：对所有可能的 a_i 都满足

$$\max_{a_i \in \mathcal{E}_i} a_i^T x \le b_i$$

等价于：

$$\bar{a}_i^T x + \|P_i^T x\| \le b_i$$

这是二阶锥约束!

SOCP 例子 2：设施选址

场景：在平面上选一个位置建设施，最小化到所有客户的总距离

客户位置：$(a_1, b_1), \ldots, (a_m, b_m)$

建模：设设施位置为 (x, y)

$$\min_{x,y} \sum_{i=1}^{m} \sqrt{(x - a_i)^2 + (y - b_i)^2}$$

引入辅助变量 t_i：

$$\begin{aligned} \min \quad & \sum_i t_i \\ \text{s.t.} \quad & \|(x - a_i, y - b_i)\|_2 \le t_i, \quad \forall i \end{aligned}$$

这是 SOCP!

SOCP 例子 3：机器人路径规划

场景：机器人从 A 到 B，避开圆形障碍物

障碍物：圆心 c_i，半径 r_i

约束：路径点 x_k 必须在所有障碍物外

$$\|x_k - c_i\| \ge r_i + \epsilon$$

问题：这是非凸的!（集合外部不是凸集）

解决方案：

- 方法 1：凸松弛 + 序列凸规划 (SCP)

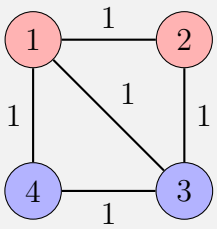
- 方法 2：整数规划（选择绕行方向）
- 方法 3：采样方法（RRT）

教训：不是所有几何问题都能直接凸化

5.4 半定规划 (SDP) 的例子

SDP 例子 1：MAX-CUT 松弛

MAX-CUT 问题：把图的顶点分成两组，最大化跨组的边数
具体例子：4 个顶点的图



切割 $\{1,2\}$ vs $\{3,4\}$
跨组边数 $= 4$

原问题：设 $x_i \in \{-1,+1\}$（+1 表示第一组，-1 表示第二组）

$$\max \sum_{(i,j)\in E} \frac{1 - x_i x_j}{2}$$

当 $x_i \neq x_j$ 时，$(1 - x_i x_j)/2 = 1$；相同时为 0
SDP 松弛：把 $x_i \in \{-1,+1\}$ 松弛为单位向量 $v_i \in \mathbb{R}^n$
设 $X_{ij} = v_i^T v_j$，则 $X \succeq 0$（Gram 矩阵）且 $X_{ii} = 1$

$$\max \sum_{(i,j)\in E} \frac{1 - X_{ij}}{2} \quad \text{s.t.} \quad X \succeq 0,\ X_{ii} = 1$$

上例的 SDP：5 条边，最大化 $\frac{1}{2}(5 - X_{12} - X_{23} - X_{34} - X_{14} - X_{13})$

SDP 例子 2：控制系统稳定性

场景：线性系统 $\dot{x} = Ax$，判断是否稳定
具体例子：

$$A = \begin{pmatrix} -1 & 2 \\ 0 & -3 \end{pmatrix}$$

Lyapunov 定理：系统稳定 $\Leftrightarrow$ 存在 $P \succ 0$ 使得 $A^T P + PA \prec 0$
设 $P = \begin{pmatrix} p_{11} & p_{12} \\ p_{12} & p_{22} \end{pmatrix}$，需要：

条件 1：$P \succ 0$

$$p_{11} > 0, \quad p_{11}p_{22} - p_{12}^2 > 0$$

条件 2：$A^T P + PA \prec 0$

$$A^T P + PA = \begin{pmatrix} -2p_{11} & 2p_{11} - 4p_{12} \\ 2p_{11} - 4p_{12} & 4p_{12} - 6p_{22} \end{pmatrix} \prec 0$$

一个解：$P = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$

验证：$A^T P + PA = \begin{pmatrix} -2 & 2 \\ 2 & -6 \end{pmatrix}$

特征值：$-4 \pm \sqrt{8} \approx -1.17, -6.83 < 0$ ✓

结论：系统稳定！（也可以直接看 A 的特征值 $-1, -3 < 0$）

SDP 例子 3：矩阵补全

场景：Netflix 评分矩阵，只观察到部分评分

具体例子：3 个用户对 4 部电影的评分（? 表示未知）

$$M = \begin{pmatrix} 5 & ? & 4 & ? \\ ? & 3 & ? & 2 \\ 4 & ? & 5 & ? \end{pmatrix}$$

假设：评分矩阵是低秩的（用户偏好可用少数因子描述）

例如：用户分为 “动作片爱好者” 和 “文艺片爱好者”，电影也分两类

核范数最小化（低秩的凸松弛）：

$$\min_X \|X\|_* \quad \text{s.t.} \quad X_{11} = 5, X_{13} = 4, X_{22} = 3, X_{24} = 2, X_{31} = 4, X_{33} = 5$$

SDP 形式：$\|X\|_* = \min \frac{1}{2}(\mathrm{tr}(W_1) + \mathrm{tr}(W_2))$

$$\text{s.t.} \quad \begin{pmatrix} W_1 & X \\ X^T & W_2 \end{pmatrix} \succeq 0$$

可能的补全结果（秩-1 近似）：

$$\hat{M} \approx \begin{pmatrix} 5 & 3 & 4 & 2.4 \\ 3 & 3 & 2.4 & 2 \\ 4 & 2.4 & 5 & 1.6 \end{pmatrix}$$

凸优化问题层次总结

更高效 ↓

- **一般凸优化**：深度学习中的凸子问题
- **SDP**：MAX-CUT、控制、矩阵补全
- **SOCP**：鲁棒优化、设施选址
- **QP**：投资组合、SVM、岭回归
- **LP**：生产计划、运输、饮食

↑ 更强大

原则：用能解决问题的**最简单**的框架

6 Lagrangian 与对偶函数

Lagrangian 函数

考虑优化问题：

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \le 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

定义 6.1 (Lagrangian)**.** ***Lagrangian* 函数**：

$$L(x, \lambda, \nu) = f(x) + \sum_{i=1}^{m} \lambda_i g_i(x) + \sum_{j=1}^{p} \nu_j h_j(x)$$

其中 $\lambda_i \ge 0$ 是不等式约束的 ***Lagrange* 乘子**，ν_j 是等式约束的乘子。

向量形式：$L(x, \lambda, \nu) = f(x) + \lambda^T g(x) + \nu^T h(x)$

对偶函数

定义 6.2 (对偶函数)**.** ***Lagrange* 对偶函数**：

$$g(\lambda, \nu) = \inf_x L(x, \lambda, \nu) = \inf_x \left[f(x) + \lambda^T g(x) + \nu^T h(x) \right]$$

关键性质：对偶函数是**凹函数**（即使原问题非凸！）
原因：g 是关于 (λ, ν) 的仿射函数的逐点下确界

对偶函数的下界性质

定理 6.3 (弱对偶). 对任意 $\lambda \geq 0$ 和任意 ν：

$$g(\lambda, \nu) \leq p^*$$

其中 p^* 是原问题的最优值。

证明：设 $\tilde{x}$ 是可行解，则 $g_i(\tilde{x}) \leq 0$，$h_j(\tilde{x}) = 0$。

$$\begin{aligned} g(\lambda, \nu) &= \inf_x L(x, \lambda, \nu) \\ &\leq L(\tilde{x}, \lambda, \nu) \\ &= f(\tilde{x}) + \underbrace{\lambda^T g(\tilde{x})}_{\leq 0} + \underbrace{\nu^T h(\tilde{x})}_{=0} \\ &\leq f(\tilde{x}) \end{aligned}$$

对所有可行解都成立，故 $g(\lambda, \nu) \leq p^*$。

7 弱对偶与强对偶

对偶问题

原问题（Primal）：

$$p^* = \min_x f(x) \quad \text{s.t.} \quad g(x) \leq 0,\ h(x) = 0$$

对偶问题（Dual）：

$$d^* = \max_{\lambda \geq 0, \nu} g(\lambda, \nu) = \max_{\lambda \geq 0, \nu} \inf_x L(x, \lambda, \nu)$$

弱对偶（总是成立）：

$$d^* \leq p^*$$

对偶间隙：$p^* - d^*$

强对偶

定义 7.1 (强对偶). 如果 $d^* = p^*$，称**强对偶**成立。

什么时候强对偶成立？

定理 7.2 (Slater 条件)**.** 对于凸优化问题，如果存在**严格可行点** $\tilde{x}$：

$$g_i(\tilde{x}) < 0, \quad i = 1, \ldots, m; \quad h_j(\tilde{x}) = 0, \quad j = 1, \ldots, p$$

则强对偶成立。

直觉：约束不能“恰好”卡在边界上，要有“余量”

直觉：对偶的价值

1. **提供下界**：即使不能解原问题，对偶给出最优值的界
2. **有时对偶更容易**：对偶问题可能有更好的结构
3. **最优性条件**：强对偶 + KKT 条件 = 最优性的充要条件
4. **敏感性分析**：对偶变量告诉我们约束的“价值”

8 KKT 条件

KKT 条件：最优性的充要条件

定理 8.1 (KKT 条件)**.** 对于凸优化问题，如果强对偶成立，则 x^* 是最优解当且仅当存在 λ^*, ν^* 使得：

1. **原始可行性**（*Primal Feasibility*）：

$$g_i(x^*) \leq 0, \quad h_j(x^*) = 0$$

2. **对偶可行性**（*Dual Feasibility*）：

$$\lambda_i^* \geq 0$$

3. **互补松弛**（*Complementary Slackness*）：

$$\lambda_i^* g_i(x^*) = 0, \quad \forall i$$

4. **稳定性**（*Stationarity*）：

$$\nabla f(x^*) + \sum_i \lambda_i^* \nabla g_i(x^*) + \sum_j \nu_j^* \nabla h_j(x^*) = 0$$

KKT 条件的推导 ★

设强对偶成立：$p^* = d^*$

Step 1：写出对偶间隙为零的条件

设 x^* 是原问题最优解，(λ^*, ν^*) 是对偶问题最优解

$$
\begin{aligned}
0 &= p^* - d^* \\
&= f(x^*) - g(\lambda^*, \nu^*) \\
&= f(x^*) - \inf_x L(x, \lambda^*, \nu^*) \\
&\geq f(x^*) - L(x^*, \lambda^*, \nu^*) \quad \text{（下确界} \leq \text{任意值）} \\
&= f(x^*) - f(x^*) - \sum_i \lambda_i^* g_i(x^*) - \sum_j \nu_j^* h_j(x^*) \\
&= -\sum_i \underbrace{\lambda_i^*}_{\geq 0} \underbrace{g_i(x^*)}_{\leq 0} - \sum_j \nu_j^* \underbrace{h_j(x^*)}_{=0} \\
&\geq 0
\end{aligned}
$$

Step 2：由 $0 \geq 0$，所有不等式必须取等

- $\inf_x L(x, \lambda^*, \nu^*) = L(x^*, \lambda^*, \nu^*)$

  $\Rightarrow x^*$ 是 $L(\cdot, \lambda^*, \nu^*)$ 的最小值点

  $\Rightarrow \nabla_x L(x^*, \lambda^*, \nu^*) = 0$ **（稳定性）**

- $\sum_i \lambda_i^* g_i(x^*) = 0$，且每项 $\lambda_i^* g_i(x^*) \leq 0$

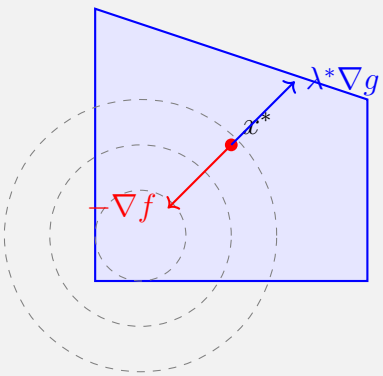
  $\Rightarrow \lambda_i^* g_i(x^*) = 0$ 对所有 i **（互补松弛）**

□

KKT 条件的直觉

稳定性：$\nabla_x L(x^*, \lambda^*, \nu^*) = 0$

在最优点，目标函数的梯度被约束的梯度“平衡”



互补松弛：$\lambda_i^* g_i(x^*) = 0$

- 如果约束不紧（$g_i(x^*) < 0$），则 $\lambda_i^* = 0$（约束“不起作用”）
- 如果 $\lambda_i^* > 0$，则约束必须紧（$g_i(x^*) = 0$）

例子：带约束的二次规划

问题：

$$\min_x \frac{1}{2}x^2 \quad \text{s.t.} \quad x \geq 1$$

转换：$g(x) = 1 - x \leq 0$
Lagrangian：$L(x, \lambda) = \frac{1}{2}x^2 + \lambda(1 - x)$
KKT 条件：

1. 原始可行：$x \geq 1$
2. 对偶可行：$\lambda \geq 0$
3. 互补松弛：$\lambda(1 - x) = 0$
4. 稳定性：$x - \lambda = 0$

求解：由稳定性 $\lambda = x$。由互补松弛，要么 $\lambda = 0$（则 $x = 0$，违反原始可行），要么 $x = 1$。
故 $x^* = 1$，$\lambda^* = 1$。

9 对偶的经济学解释

影子价格：约束的边际价值

考虑带扰动的优化问题：

$$\begin{aligned} p^*(\epsilon) = \min_x \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \leq \epsilon_i, \quad i = 1, \ldots, m \end{aligned}$$

当 $\epsilon = 0$ 时，就是原问题，最优值为 $p^*(0)$
问题：如果把第 i 个约束放松一点（$\epsilon_i > 0$），最优值会改善多少？

影子价格定理

定理 9.1 (影子价格)**.** *在最优解处，对偶变量 λ_i^* 满足：*

$$\lambda_i^* = -\frac{\partial p^*}{\partial \epsilon_i}\bigg|_{\epsilon=0}$$

直觉：λ_i^* 告诉你“放松约束 i 一个单位，目标函数能改善多少”
近似公式：当 ϵ 很小时

$$p^*(\epsilon) \approx p^*(0) - \sum_{i=1}^{m} \lambda_i^* \epsilon_i$$

具体例子：生产问题

问题：最小化成本，受资源约束

$$\begin{aligned} \min_x \quad & 2x_1 + 3x_2 \quad \text{（成本）} \\ \text{s.t.} \quad & x_1 + x_2 \geq 10 \quad \text{（产量需求）} \\ & x_1 \leq 8 \quad \text{（机器 A 产能）} \\ & x_2 \leq 6 \quad \text{（机器 B 产能）} \\ & x_1, x_2 \geq 0 \end{aligned}$$

转换为标准形式（$g_i(x) \leq 0$）：

$$\begin{aligned} g_1(x) &= -x_1 - x_2 + 10 \leq 0 \quad \text{（产量约束）} \\ g_2(x) &= x_1 - 8 \leq 0 \quad \text{（机器 A）} \\ g_3(x) &= x_2 - 6 \leq 0 \quad \text{（机器 B）} \end{aligned}$$

最优解：$x_1^* = 4$，$x_2^* = 6$，成本 $= 2(4) + 3(6) = 26$

影子价格的计算与解释

KKT 条件给出：$\lambda_1^* = 2$，$\lambda_2^* = 0$，$\lambda_3^* = 1$
解释：
约束 1（产量需求 ≥ 10）：$\lambda_1^* = 2$

- 如果只需要生产 9 件（放松 1 单位）
- 新问题：$g_1(x) = -x_1 - x_2 + 10 \leq 1$
- 成本减少约 $\lambda_1^* \times 1 = 2$ 元

约束 2（机器 A 产能 ≤ 8）：$\lambda_2^* = 0$

- 当前 $x_1^* = 4 < 8$，产能没用完
- 增加机器 A 产能没有价值！

约束 3（机器 B 产能 ≤ 6）：$\lambda_3^* = 1$

- 当前 $x_2^* = 6$，产能用尽
- 如果机器 B 产能增加到 7（放松 1 单位）
- 新问题：$g_3(x) = x_2 - 6 \leq 1$
- 成本减少约 $\lambda_3^* \times 1 = 1$ 元

验证：放松约束后重新求解

原问题：机器 B 产能 ≤ 6

- 最优解：$x_1^* = 4$，$x_2^* = 6$
- 成本：26

放松后：机器 B 产能 ≤ 7

- 新最优解：$x_1^* = 3$，$x_2^* = 7$
- 新成本：$2(3) + 3(7) = 27$?

等等，成本增加了？让我们重新思考...
注意：我们是在**最小化**成本，放松产能约束不一定降低成本！
正确理解：

- $\lambda_3^* = 1$ 意味着约束 "值 1 元"
- 如果**收紧**约束（机器 B 只能生产 5 件），成本会**增加**约 1 元
- 验证：$x_1 = 5, x_2 = 5$，成本 $= 2(5) + 3(5) = 25$...

重新审视：影子价格的符号取决于问题的设定方式！

影子价格的正确理解

关键公式：

$$p^*(\epsilon) \approx p^*(0) - \lambda^{*T}\epsilon$$

对于最小化问题：

- $\epsilon_i > 0$（放松约束）$\Rightarrow$ 可行域变大 $\Rightarrow$ p^* 可能变小
- $\lambda_i^* \geq 0$，所以 $-\lambda_i^*\epsilon_i \leq 0$

经济学直觉：

- $\lambda_i^* =$ 约束 i 的 "影子价格"
- $=$ 你愿意为 "放松约束一个单位" 支付的最高价格
- $=$ 约束的边际成本

实际应用：

- 生产计划：应该购买哪种资源？（看哪个 λ_i^* 最大）
- 电力市场：边际发电成本
- 物流优化：运力瓶颈在哪？

互补松弛的经济学意义

互补松弛：$\lambda_i^* g_i(x^*) = 0$
翻译成人话：

情况	数学	经济学
资源有剩余	$g_i(x^*) < 0$	供过于求
$\Rightarrow$ 价格为零	$\lambda_i^* = 0$	免费资源无价值
资源有价值	$\lambda_i^* > 0$	稀缺资源
$\Rightarrow$ 资源用尽	$g_i(x^*) = 0$	供不应求

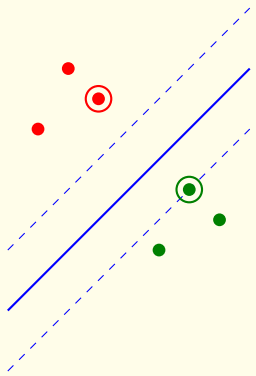
直觉：只有**稀缺**的资源才有价格！

直觉：SVM 中的支持向量

SVM 对偶问题的变量 α_i 就是 Lagrange 乘子
互补松弛：$\alpha_i(1 - y_i(w^T x_i + b)) = 0$

- $\alpha_i = 0$：点 x_i 不在边界上（不是支持向量）
- $\alpha_i > 0$：点 x_i 恰好在边界上（是支持向量）

结论：只有**支持向量**对决策边界有贡献！



10 机器学习视角：凸优化的现代应用

机器学习 = 优化

几乎所有机器学习都可以写成：

$$\min_\theta \underbrace{\frac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(x_i), y_i)}_{\text{经验风险}} + \underbrace{\lambda R(\theta)}_{\text{正则化}}$$

模型	损失函数 ℓ	正则化 R
线性回归	$(y - w^T x)^2$	$\|w\|_2^2$（岭回归）
Logistic 回归	$\log(1 + e^{-y \cdot w^T x})$	$\|w\|_2^2$ 或 $\|w\|_1$
SVM	$\max(0, 1 - y \cdot w^T x)$	$\|w\|_2^2$
神经网络	交叉熵	$\|W\|_F^2$（weight decay）

关键问题：哪些是凸的？哪些不是？

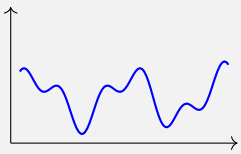
凸 vs 非凸机器学习

	凸	**非凸**
模型	线性回归、Logistic、SVM Lasso、岭回归	神经网络、矩阵分解 深度学习
优点	全局最优保证 理论完善	表达能力强 能学复杂模式
缺点	表达能力有限 只能学线性边界	可能陷入局部最优 理论不完善
优化方法	内点法、坐标下降 精确解或收敛保证	SGD、Adam 启发式，实践有效

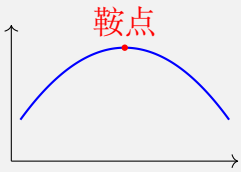
为什么深度学习“不需要”凸优化？

传统观点：非凸 = 噩梦（局部最优、鞍点）
现代发现：深度网络的损失景观有特殊结构

1. **过参数化**：参数远多于数据，存在很多全局最优
2. **鞍点主导**：高维空间中，局部最优很少，大多是鞍点
3. **好的局部最优**：即使是局部最优，泛化性能也很好
4. **隐式正则化**：SGD 本身倾向于找“平坦”的最优解



低维：很多局部最优



高维：鞍点主导

但是：凸优化的思想仍然重要！

凸优化思想在深度学习中的应用

1. 分层凸松弛
神经网络训练非凸，但每一层固定其他层后是凸的：

- 固定 W_2，优化 W_1：凸问题
- 交替优化：坐标下降的思想

2. 正则化 = 凸约束
L_2 正则化：$\min \text{Loss} + \lambda \|w\|_2^2$
等价于约束优化：$\min \text{Loss}$ s.t. $\|w\|_2 \le R$
对偶变量 λ = 约束的影子价格！
3. 对偶在 Kernel 方法中的核心作用
SVM 对偶：

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

核技巧（Kernel Trick）完全依赖对偶形式！
4. KKT 条件 → 最优性检验
即使用 SGD 训练，也可以用 KKT 残差判断是否收敛

现代机器学习中的凸子问题

很多非凸问题包含凸子问题：

问题	凸子问题
神经网络训练	最后一层是凸的（固定特征）
EM 算法	M-step 通常是凸的
矩阵分解	固定一个因子，另一个是凸的
变分推断	ELBO 优化常是凸的
强化学习	Policy evaluation 是线性方程
对抗训练	内层 max 可能是凸的

启示：理解凸优化 = 理解复杂系统的 “积木块”

Logistic 回归的凸性证明 ★

损失函数：

$$L(w) = \sum_{i=1}^{n} \log(1 + e^{-y_i w^T x_i})$$

要证：$L(w)$ 是凸函数

证明：设 $\ell(z) = \log(1 + e^{-z})$，则 $L(w) = \sum_i \ell(y_i w^T x_i)$

Step 1：证明 $\ell(z)$ 是凸的

$$\ell'(z) = \frac{-e^{-z}}{1 + e^{-z}} = \frac{-1}{1 + e^{z}} = -(1 - \sigma(z))$$

$$\ell''(z) = \sigma(z)(1 - \sigma(z)) \geq 0 \quad \checkmark$$

Step 2：$\ell(y_i w^T x_i)$ 是 w 的凸函数

因为 $y_i w^T x_i$ 是 w 的仿射函数，凸函数复合仿射仍是凸

Step 3：凸函数的和是凸函数 □

加上正则化：$L(w) + \lambda \|w\|_2^2$ 是**强凸**的（$\lambda > 0$）

11 MDP 与线性规划 ★

从实际问题出发

问题 1：库存管理

你经营一家书店，每天要决定进多少货：

- 进多了：库存成本高，书可能卖不掉
- 进少了：缺货损失销售机会

- 需求是随机的（今天可能卖 10 本，也可能卖 50 本）

问题 2：机器维护
你管理一台机器，每天要决定是否维护：

- 机器有 “好”“中”“差” 三种状态
- 不维护：省钱，但机器可能变差
- 维护：花钱，但机器变好
- 机器太差会坏掉，损失更大

问题 3：走迷宫
机器人在网格中找出口：

- 每一步可以上下左右移动
- 有些格子是障碍物
- 目标：用最少步数到达出口

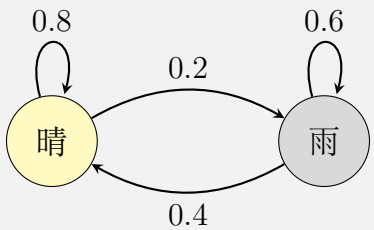
共同特点：状态 → 决策 → 随机转移 → 新状态 → ...

什么是马尔可夫性？

马尔可夫性（Markov Property）：未来只依赖于现在，与过去无关

$$P(s_{t+1}|s_t, s_{t-1}, \ldots, s_0) = P(s_{t+1}|s_t)$$

例子：天气



明天是否下雨，只取决于今天的天气，不取决于昨天、前天...
马尔可夫链：状态按概率转移，没有决策
马尔可夫决策过程 (MDP)：加入决策！你可以选择动作影响转移

MDP 的正式定义

MDP $= (S, A, P, R, \gamma)$

- S：状态空间（有限集合 $\{1, 2, \ldots, n\}$）
- A：动作空间（有限集合）
- $P(s'|s, a)$：转移概率（在状态 s 执行动作 a 后到达 s' 的概率）
- $R(s, a)$：即时奖励（在状态 s 执行动作 a 获得的奖励）
- $\gamma \in [0, 1)$：折扣因子（未来奖励的衰减）

策略：$\pi : S \to A$，告诉你在每个状态该做什么
目标：找最优策略 π^*，最大化累积折扣奖励

$$\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right]$$

为什么要折扣？

- 今天的 1 元比明天的 1 元更值钱
- 数学上保证级数收敛
- γ 接近 1：更有远见；γ 接近 0：更短视

11.1 具体例子：机器维护问题

机器维护 MDP

状态：$S = \{\text{好}, \text{中}, \text{差}\}$
动作：$A = \{\text{不维护}, \text{维护}\}$
转移概率（不维护时机器会慢慢变差）：

$P(\cdot\|s, \text{不维护})$	好	中	差
好	0.7	0.3	0
中	0	0.6	0.4
差	0	0	1

$P(\cdot\|s, \text{维护})$	好	中	差
好	1	0	0
中	0.5	0.5	0
差	0.2	0.6	0.2

奖励（好机器生产多，维护要花钱）：

$R(s,a)$	不维护	维护
好	10	7
中	5	2
差	-1	-4

问题：应该什么时候维护?

机器维护：两种策略对比

策略 1：从不维护

- 短期省钱
- 长期机器变差，损失大

策略 2：状态 “中” 或 “差” 时维护

- 维护有成本
- 但机器保持好状态，长期收益高

哪个更好？需要计算！
设 $\gamma = 0.9$，我们来求最优策略...

11.2 Bellman 方程

值函数：衡量状态的好坏

定义 11.1 (状态值函数). 策略 π 下，从状态 s 出发的期望累积奖励：

$$V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t)) \mid s_0 = s\right]$$

直觉：$V^\pi(s)$ 告诉你 “从状态 s 出发，按策略 π 走，期望能拿多少奖励”
最优值函数：

$$V^*(s) = \max_\pi V^\pi(s)$$

这是所有策略中能达到的最大值

Bellman 方程：递归结构

定理 11.2 (Bellman 期望方程).

$$V^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} P(s'|s, \pi(s)) V^\pi(s')$$

翻译：从 s 出发的总价值 = 立即奖励 + 折扣 × 下一状态的期望价值

定理 11.3 (Bellman 最优方程)**.**

$$V^*(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^*(s') \right]$$

翻译：最优价值 = 选择最好的动作，获得立即奖励 + 折扣 × 下一状态的最优价值

机器维护：计算值函数

设 $\gamma = 0.9$，Bellman 最优方程：

状态 "好"：

$$V^*(\text{好}) = \max \begin{cases} 10 + 0.9(0.7V^*(\text{好}) + 0.3V^*(\text{中})) & \text{不维护} \\ 7 + 0.9 \cdot V^*(\text{好}) & \text{维护} \end{cases}$$

状态 "中"：

$$V^*(\text{中}) = \max \begin{cases} 5 + 0.9(0.6V^*(\text{中}) + 0.4V^*(\text{差})) & \text{不维护} \\ 2 + 0.9(0.5V^*(\text{好}) + 0.5V^*(\text{中})) & \text{维护} \end{cases}$$

状态 "差"：

$$V^*(\text{差}) = \max \begin{cases} -1 + 0.9 \cdot V^*(\text{差}) & \text{不维护} \\ -4 + 0.9(0.2V^*(\text{好}) + 0.6V^*(\text{中}) + 0.2V^*(\text{差})) & \text{维护} \end{cases}$$

这是一个非线性方程组（因为有 max）... 如何求解?

11.3 MDP 的线性规划形式

关键观察：把 max 变成不等式

Bellman 最优方程：

$$V^*(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^*(s') \right]$$

等价于：$V^*(s)$ 是满足以下条件的**最小**的 V

对所有状态 s 和动作 a：

$$V(s) \geq R(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s')$$

为什么？

- V^* 显然满足这些不等式（因为 max $\geq$ 任何一项）
- 如果 V 满足所有不等式且 $V < V^*$，则存在某个 s 使得
  $V(s) < V^*(s) = R(s,a^*) + \gamma \sum_{s'} P(s'|s,a^*)V^*(s')$
  但 $V(s) \geq R(s,a^*)+\gamma \sum_{s'} P(s'|s,a^*)V(s') \geq R(s,a^*)+\gamma \sum_{s'} P(s'|s,a^*)V^*(s')$
  矛盾！

MDP 的 LP 形式

定理 11.4 (MDP 的线性规划)**.** *最优值函数 V^* 是以下 LP 的解：*

$$\begin{aligned} \min_V \quad & \sum_s \mu(s)V(s) \\ s.t. \quad & V(s) \geq R(s,a) + \gamma \sum_{s'} P(s'|s,a)V(s'), \quad \forall s \in S, \forall a \in A \end{aligned}$$

其中 $\mu(s) > 0$ 是任意正的权重。

这是一个标准的 LP！

- 变量：$V(s)$，共 $|S|$ 个
- 约束：每对 (s,a) 一个，共 $|S| \times |A|$ 个
- 目标：线性
- 约束：线性

可以用单纯形法或内点法求解！

机器维护问题的 LP

状态：好 (G)、中 (M)、差 (B)，动作：不维护 (N)、维护 (R)
变量：V_G, V_M, V_B
LP（取 $\mu = (1/3, 1/3, 1/3)$）：

$$\min \frac{1}{3}(V_G + V_M + V_B)$$

约束（6 个，每个 (s,a) 一个）：

$$
\begin{aligned}
V_G &\geq 10 + 0.9(0.7V_G + 0.3V_M) && \text{(好, 不维护)} \\
V_G &\geq 7 + 0.9V_G && \text{(好, 维护)} \\
V_M &\geq 5 + 0.9(0.6V_M + 0.4V_B) && \text{(中, 不维护)} \\
V_M &\geq 2 + 0.9(0.5V_G + 0.5V_M) && \text{(中, 维护)} \\
V_B &\geq -1 + 0.9V_B && \text{(差, 不维护)} \\
V_B &\geq -4 + 0.9(0.2V_G + 0.6V_M + 0.2V_B) && \text{(差, 维护)}
\end{aligned}
$$

用 CVXPY 求解

```python
import cvxpy as cp
import numpy as np

# 变量
V = cp.Variable(3)  # V[0]=V_G, V[1]=V_M, V[2]=V_B
gamma = 0.9

# 目标
obj = cp.Minimize(cp.sum(V) / 3)

# 约束
constraints = [
    # 状态 好
    V[0] >= 10 + gamma * (0.7*V[0] + 0.3*V[1]),  # 不维护
    V[0] >= 7 + gamma * V[0],                     # 维护
    # 状态 中
    V[1] >= 5 + gamma * (0.6*V[1] + 0.4*V[2]),   # 不维护
    V[1] >= 2 + gamma * (0.5*V[0] + 0.5*V[1]),   # 维护
    # 状态 差
    V[2] >= -1 + gamma * V[2],                    # 不维护
    V[2] >= -4 + gamma * (0.2*V[0] + 0.6*V[1] + 0.2*V[2])  # 维护
]

prob = cp.Problem(obj, constraints)
prob.solve()
print(f"V* = {V.value}")  # [73.8, 64.5, 47.2] 大约
```

从值函数恢复最优策略

求解 LP 得到：$V_G^* \approx 73.8$，$V_M^* \approx 64.5$，$V_B^* \approx 47.2$
最优策略：选择使 Bellman 方程取等的动作
状态 "好"：

- 不维护：$10 + 0.9(0.7 \times 73.8 + 0.3 \times 64.5) \approx 73.8$ ✓
- 维护：$7 + 0.9 \times 73.8 \approx 73.4$

$\Rightarrow$ **不维护**
状态 "中"：

- 不维护：$5 + 0.9(0.6 \times 64.5 + 0.4 \times 47.2) \approx 56.8$
- 维护：$2 + 0.9(0.5 \times 73.8 + 0.5 \times 64.5) \approx 64.5$ ✓

$\Rightarrow$ **维护**
状态 "差"：

- 不维护：$-1 + 0.9 \times 47.2 \approx 41.5$
- 维护：$-4 + 0.9(0.2 \times 73.8 + 0.6 \times 64.5 + 0.2 \times 47.2) \approx 47.2$ ✓

$\Rightarrow$ **维护**
最优策略：好 $\rightarrow$ 不维护，中 $\rightarrow$ 维护，差 $\rightarrow$ 维护

11.4 LP 对偶与占用度量

MDP 的对偶 LP

原问题有对偶！引入对偶变量 $\rho(s,a) \geq 0$

定理 11.5 (MDP 的对偶 LP)**.**

$$\begin{aligned} \max_{\rho} \quad & \sum_{s,a} \rho(s,a) R(s,a) \\ s.t. \quad & \sum_{a} \rho(s,a) = \mu(s) + \gamma \sum_{s',a'} \rho(s',a') P(s|s',a'), \quad \forall s \\ & \rho(s,a) \geq 0 \end{aligned}$$

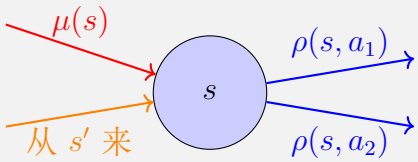
$\rho(s,a)$ **的含义**：**占用度量**（Occupancy Measure）
$\rho(s,a) =$ 在最优策略下，处于状态 s 并执行动作 a 的（折扣）频率

对偶约束的直觉：流量守恒

约束：$\sum_a \rho(s,a) = \mu(s) + \gamma \sum_{s',a'} \rho(s',a')P(s|s',a')$

翻译：

$$\underbrace{\sum_a \rho(s,a)}_{\text{从}s\text{ 流出}} = \underbrace{\mu(s)}_{\text{初始流入}} + \underbrace{\gamma \sum_{s',a'} \rho(s',a')P(s|s',a')}_{\text{从其他状态转移来}}$$



流入 = 流出

这就是网络流的守恒定律！

原问题 vs 对偶问题

	原问题	**对偶问题**
变量	$V(s)$：值函数	$\rho(s,a)$：占用度量
数量	$\lvert S\rvert$ 个	$\lvert S\rvert \times \lvert A\rvert$ 个
约束	Bellman 不等式	流量守恒
目标	最小化总值	最大化总奖励
视角	“状态有多好？”	“应该怎么做？”

强对偶：两个 LP 的最优值相等！（凸优化的对偶理论）

从对偶解恢复策略：

$$\pi^*(a|s) = \frac{\rho^*(s,a)}{\sum_{a'} \rho^*(s,a')}$$

互补松弛在 MDP 中的意义

KKT 互补松弛：$\rho^*(s,a) \cdot [V^*(s) - R(s,a) - \gamma \sum_{s'} P(s'|s,a)V^*(s')] = 0$

翻译：

- 如果 $\rho^*(s,a) > 0$（动作 a 在状态 s 被执行）

  $\Rightarrow$ Bellman 不等式取等（a 是最优动作）

- 如果动作 a 不是最优的（Bellman 不等式严格）

  $\Rightarrow \rho^*(s,a) = 0$（这个动作不会被执行）

经济学解释：

- $V^*(s)$ = 状态 s 的 "影子价格"
- 只有 "物有所值" 的动作才会被使用
- 次优动作的 "使用量" 为零

直觉：MDP-LP 总结

核心结论：

- MDP 的最优策略可以通过 LP 精确求解
- 原问题变量是值函数，对偶变量是占用度量
- LP 对偶揭示了 "评估" 与 "决策" 的深层联系

为什么这很重要？

- 把决策问题转化为我们熟悉的凸优化
- KKT 条件给出最优性的刻画
- 影子价格、互补松弛都有自然解释

局限性：

- 需要知道完整的 MDP 模型（P, R）
- 状态空间大时（如围棋 10^{170} 状态），LP 不可行
- 实际中常用近似方法（值迭代、策略迭代、函数逼近）

但 LP 视角提供了理论基础！

12 GPU 改变现代优化

经典优化 vs 现代优化

经典优化（1960s-2000s）：

- 内点法、单纯形法
- 每步计算量大，但步数少
- 适合中等规模问题（$n < 10^5$）

- 需要精确的二阶信息（Hessian）

现代优化（2010s-）：

- 随机梯度下降（SGD）及其变体
- 每步计算量小，但步数多
- 适合超大规模问题（$n > 10^9$）
- 只需要一阶信息（梯度）

为什么变了？

GPU 视角：GPU 的出现改变了游戏规则

内点法的瓶颈：
每步需要解线性系统 $H\Delta x = -g$，其中 H 是 Hessian

- 形成 H：$O(n^2)$ 存储
- 求解线性系统：$O(n^3)$ 或稀疏情况 $O(n^{1.5})$
- **很难并行化**

梯度下降的优势：
每步只需计算 $x \leftarrow x - \alpha \nabla f(x)$

- 只需 $O(n)$ 存储
- 计算梯度：可以高度并行化
- **完美匹配 GPU 架构**

数值对比

问题：$n = 10^6$ 变量的凸优化

方法	每步复杂度	收敛步数	实际时间
内点法 (CPU)	$O(n^3)$	~ 50	几天（内存不够）
梯度下降 (CPU)	$O(n)$	$\sim 10^4$	几小时
梯度下降 (GPU)	$O(n)$ 并行	$\sim 10^4$	**几分钟**

深度学习的选择：

神经网络参数量：$n \sim 10^9$（GPT-3 有 1750 亿参数）
内点法？绝对不可能。只能用 SGD + GPU。

批量优化：GPU 的超能力

场景：需要解 1000 个独立的小优化问题
传统方法（顺序）：

- 每个问题 0.1 秒
- 总时间：$1000 \times 0.1 = 100$ 秒

GPU 方法（并行）：

- 1000 个问题同时求解
- 总时间：≈ 0.1 秒（几乎和 1 个问题一样）

加速比：$\sim 1000\times$
应用：

- 强化学习：同时模拟 1000 个环境
- 超参数搜索：同时训练 1000 个模型
- 蒙特卡洛：同时采样 1000 条路径

直觉：理论 vs 实践的张力

理论说：
凸优化用内点法，$O(\sqrt{n})$ 步收敛，每步 $O(n^3)$
实践说：
管他收敛多少步，能在 GPU 上跑、能处理 10^9 变量才是王道
调和：

- 小问题（$n < 10^4$）：内点法仍是王者
- 大问题（$n > 10^6$）：一阶方法 + GPU
- 结构化问题：针对性算法（如 ADMM）

本课程：两种视角都要学，知道何时用哪种

13 实验

13.1 实验 2a：CVXPY 求解 SVM 对偶

实验：观察 KKT 条件与支持向量

目标：

- 用 CVXPY 求解 SVM 对偶问题
- 观察互补松弛条件
- 识别支持向量

SVM 原始问题：

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1$$

SVM 对偶问题：

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j x_i^T x_j \quad \text{s.t.} \quad \alpha_i \geq 0, \sum_i \alpha_i y_i = 0$$

实验代码框架

```python
import cvxpy as cp
import numpy as np

# 生成数据
np.random.seed(42)
n_samples = 50
X_pos = np.random.randn(n_samples//2, 2) + [2, 2]
X_neg = np.random.randn(n_samples//2, 2) + [-2, -2]
X = np.vstack([X_pos, X_neg])
y = np.array([1]*(n_samples//2) + [-1]*(n_samples//2))

# 对偶问题
n = len(y)
alpha = cp.Variable(n)
K = (X @ X.T) * np.outer(y, y)  # Gram 矩阵

objective = cp.Maximize(cp.sum(alpha) - 0.5 * cp.quad_form(alpha
    , K))
constraints = [alpha >= 0, alpha @ y == 0]
```

```python
prob = cp.Problem(objective, constraints)
prob.solve()

# 找支持向量
alpha_val = alpha.value
support_vectors = np.where(alpha_val > 1e-5)[0]
print(f"支持向量数量: {len(support_vectors)}")
print(f"支持向量的 alpha: {alpha_val[support_vectors]}")
```

观察互补松弛

KKT 互补松弛：$\alpha_i(1 - y_i(w^T x_i + b)) = 0$

```python
# 从对偶解恢复原始变量
w = sum(alpha_val[i] * y[i] * X[i] for i in range(n))
# 从支持向量计算 b
sv_idx = support_vectors[0]
b = y[sv_idx] - w @ X[sv_idx]

# 验证互补松弛
for i in range(n):
    margin = y[i] * (w @ X[i] + b)
    slack = alpha_val[i] * (1 - margin)
    if alpha_val[i] > 1e-5:
        print(f"点 {i}: alpha={alpha_val[i]:.4f}, "
              f"margin={margin:.4f}, slack={slack:.6f}")
```

预期结果：

- $\alpha_i > 0$ 的点：margin ≈ 1（在边界上）
- $\alpha_i = 0$ 的点：margin > 1（远离边界）

13.2 实验 2b：GPU vs CPU 凸优化

实验：规模与方法的权衡

目标：比较不同规模下的最优策略

规模	推荐方法	原因
小 ($n \sim 100$)	CVXPY 内点法	精确、快速
中 ($n \sim 10^4$)	视情况	过渡区域
大 ($n \sim 10^5+$)	GPU 梯度下降	内点法内存爆炸

实验代码：小规模 QP

```python
import cvxpy as cp
import numpy as np
import time

# 小规模 QP: n = 100
n = 100
np.random.seed(42)
P = np.random.randn(n, n)
P = P.T @ P + 0.1 * np.eye(n)  # 确保正定
q = np.random.randn(n)

x = cp.Variable(n)
objective = cp.Minimize(0.5 * cp.quad_form(x, P) + q @ x)
constraints = [x >= 0]
prob = cp.Problem(objective, constraints)

start = time.time()
prob.solve()
print(f"CVXPY (n={n}): {time.time() - start:.4f} 秒")
print(f"最优值: {prob.value:.4f}")
```

预期：内点法在毫秒级完成

实验代码：大规模 QP

```python
import torch

# 大规模 QP: n = 50000
n = 50000
device = 'cuda' if torch.cuda.is_available() else 'cpu'

P = torch.randn(n, n, device=device)
```

```
P = P.T @ P + 0.1 * torch.eye(n, device=device)
q = torch.randn(n, device=device)

# 投影梯度下降
x = torch.zeros(n, device=device, requires_grad=True)
optimizer = torch.optim.Adam([x], lr=0.01)

start = time.time()
for i in range(1000):
    loss = 0.5 * x @ P @ x + q @ x
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    with torch.no_grad():
        x.clamp_(min=0)  # 投影到 x >= 0

    if i % 200 == 0:
        print(f"Step {i}: loss = {loss.item():.4f}")

print(f"GPU 梯度下降 (n={n}): {time.time() - start:.2f} 秒")
```

实验代码：批量优化

```
# 批量优化：同时解 1000 个独立的小 QP
batch_size = 1000
n = 100

# 生成 1000 个不同的问题
P_batch = torch.randn(batch_size, n, n, device=device)
P_batch = P_batch.transpose(-1, -2) @ P_batch
P_batch = P_batch + 0.1 * torch.eye(n, device=device)
q_batch = torch.randn(batch_size, n, device=device)

x_batch = torch.zeros(batch_size, n, device=device,
                      requires_grad=True)
optimizer = torch.optim.Adam([x_batch], lr=0.1)

start = time.time()
```

```python
for i in range(500):
    # 批量计算所有问题的目标函数
    loss = 0.5 * torch.einsum('bi,bij,bj->b',
                              x_batch, P_batch, x_batch)
    loss = loss + torch.einsum('bi,bi->b', q_batch, x_batch)
    loss = loss.sum()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    with torch.no_grad():
        x_batch.clamp_(min=0)

print(f"批量优化 1000 个问题: {time.time() - start:.2f} 秒")
```

预期：1000 个问题的时间 $\approx$ 1 个问题的时间

14 本周总结

核心结论：数学定理总结

凸性判定（会证明 ★）：

- 一阶条件：$f(y) \geq f(x) + \nabla f(x)^T(y-x) \Leftrightarrow f$ 凸
- 二阶条件：$\nabla^2 f(x) \succeq 0$ 对所有 $x \Leftrightarrow f$ 凸
- 强凸：$\nabla^2 f(x) \succeq mI$，$m > 0$

对偶理论：

- 弱对偶（总成立）：$d^* \leq p^*$
- 强对偶（Slater 条件）：$d^* = p^*$
- 对偶间隙：$p^* - d^* \geq 0$

KKT 条件（会推导 ★）：

1. 原始可行：$g_i(x^*) \leq 0$，$h_j(x^*) = 0$
2. 对偶可行：$\lambda_i^* \geq 0$
3. 互补松弛：$\lambda_i^* g_i(x^*) = 0$
4. 稳定性：$\nabla_x L = 0$

核心结论：现代视角

凸优化在机器学习中的地位：

- 经典 ML（SVM、Logistic、Lasso）：凸优化是核心
- 深度学习：非凸，但凸优化思想仍然重要
- 子问题：很多非凸问题包含凸子问题

理论 vs 实践：

- 小规模（$n < 10^4$）：内点法，精确解
- 大规模（$n > 10^6$）：一阶方法 + GPU
- 批量问题：GPU 并行，1000 个 $\approx$ 1 个

对偶的三重价值：

1. 提供下界（最优性证书）
2. 影子价格（敏感性分析）
3. 核技巧（SVM、Kernel 方法）

核心结论：带走的核心技能

概念层面：

- 能判断一个优化问题是否是凸的
- 理解 "凸 = 局部即全局" 的意义
- 知道 KKT 条件的四个组成部分

计算层面：

- 能写出简单问题的 Lagrangian
- 能用 KKT 条件手算小问题
- 能用 CVXPY 求解中等规模凸优化

直觉层面：

- 互补松弛 = 稀缺资源才有价格
- 对偶变量 = 约束的边际价值
- 支持向量 = 对偶变量非零的点

15 思考题

思考题

1. 证明：如果 f 和 g 都是凸函数，则 $\max\{f(x), g(x)\}$ 也是凸函数。

2. 为什么凸优化的等式约束只能是仿射的？举一个反例说明非仿射等式约束会破坏凸性。

3. 考虑问题 $\min x^2$ s.t. $x \geq 2$。

(a) 写出 KKT 条件

(b) 求解最优解和最优 Lagrange 乘子

(c) 验证互补松弛条件

4. 在 SVM 中，如果所有训练样本都正确分类且远离边界，会发生什么？这时有多少支持向量？

5. 设计实验：对于 $n = 1000, 5000, 10000, 50000$ 的 QP 问题，分别用 CVXPY 内点法和 GPU 梯度下降求解，画出 "问题规模 vs 求解时间" 的曲线。在什么规模

下 GPU 开始占优势?